

Sketchnote: Sketch-Based Visualization of Problem Decomposition in Block-Based Programming

Seung Hyeon Han

hsh0908y@kaist.ac.kr

KAIST

Department of Industrial Design

Daejeon, Republic of Korea

Woohun Lee

woohun.lee@kaist.ac.kr

KAIST

Department of Industrial Design

Daejeon, Republic of Korea

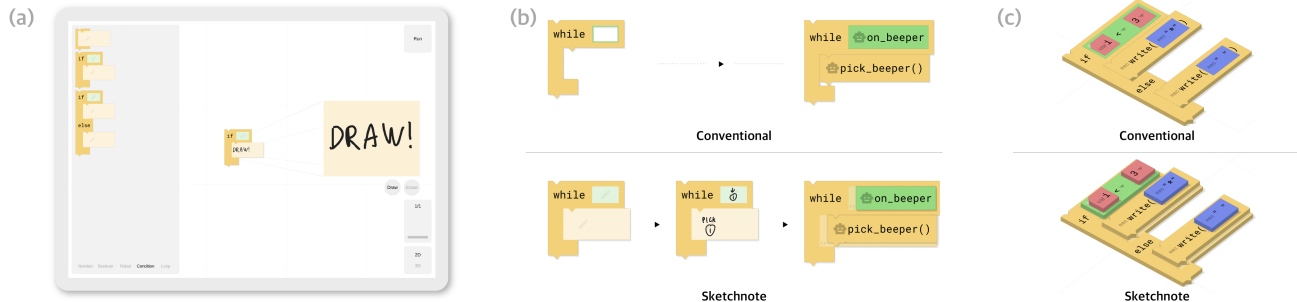


Figure 1: (a) Basic UI layout of Sketchnote. Comparison between conventional block-based programming and Sketchnote on (b) the attachment slot and (c) the layer structure. Sketches can be drawn and reinterpreted before putting code blocks.

ABSTRACT

Block-based programming effectively supports processes based on the syntactic and conceptual knowledge of programming; however, its effectiveness is limited to processes that require strategic knowledge. To resolve the problem, we present Sketchnote, which visualizes the problem decomposition process with sketching and multi-layered structure in block-based programming. Sketchnote allows programmers to sketch in the block attachment slots before inserting the actual code blocks. The multi-layer structure of Sketchnote also visualizes the hierarchy of code while extending the brick metaphor of block-based programming.

CCS CONCEPTS

• **Software and its engineering** → **Visual languages**; *Integrated and visual development environments*.

KEYWORDS

block-based programming, visual programming languages, problem decomposition, sketching

ACM Reference Format:

Seung Hyeon Han and Woohun Lee. 2023. Sketchnote: Sketch-Based Visualization of Problem Decomposition in Block-Based Programming. In *The 36th Annual ACM Symposium on User Interface Software and Technology*

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

UIST '23 Adjunct, October 29–November 01, 2023, San Francisco, CA, USA

© 2023 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0096-5/23/10.

<https://doi.org/10.1145/3586182.3616654>

(UIST '23 Adjunct), October 29–November 01, 2023, San Francisco, CA, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3586182.3616654>

1 INTRODUCTION

Block-based programming is one of the most successful visual programming approaches and is widely used in educational settings [7, 8]. This method simplifies text-based programming by breaking down text codes into puzzle-like code snippets. It provides a more accessible and intuitive way for beginners to learn programming and can help them develop fundamental programming concepts. It also stimulates interest in programming with toy-like forms and methods, especially for children.

Block-based visualization has undoubtedly reduced many of the difficulties that novice programmers encounter, but not all the challenges have been addressed. Knowledge and difficulties in programming can be categorized as syntactic, conceptual, or strategic ones [1, 3, 5]. Block-based programming has helped mitigate syntactic and conceptual difficulties: for example liberation from syntax errors, providing visual affordance to combine blocks of the correct type, minimizing commands, using terms closer to natural language, or providing icons related to the programming concept [2]. However, improvements in the strategic aspects do not yet seem substantial.

To address these problems, we propose an approach to visualizing problem decomposition through sketching. We aim to expand the visualization factors in block-based programming beyond the traditional focus on syntactic and conceptual factors, and into the realm of strategic factors. Our approach, called Sketchnote, allows programmers to first draw an abstract sketch on the attachment slot of the block, and replace it later with a proper code block. Additionally, we modified the block structure from single-layer to

multi-layer to facilitate the sketch-and-replace process and visually represent the structural hierarchy of the program.

2 SKETCHNOTE

2.1 Sketching

To achieve the purpose, we first modified the structure and combination method of blocks. In most block-based programming languages, the attachment slot of the expression block is blank, and that of the statement block has a C shape, which we suggest calling **holes** and **dents**. In Sketchnote, as shown in Figure 1(b), the holes and dents are modified into **sketch areas**. With this, the contents to be implemented can be drawn in the area before attaching the next block.

One of the great advantages of sketching is that they can express things abstractly. Even when the programmer cannot explain the details in the initial stage of programming, it would be easier to express them abstractly by drawing sketches. In addition, sketches are suitable for expressing the movements of toys and robots, which are widely used in child programming education. It provides an easier way to write comments on tablet computers, which are widely used by children, because typing texts on them is sometimes hard. The activity of drawing sketches itself is also appealing, particularly for young children. This can be an advantage compared to writing comments, which is necessary but often considered troublesome.

2.1.1 Folding. When a block is first placed on a sketch, the sketch area expands to the side and the block is displayed next to the sketch. The sketch areas can be **folded** (or **unfolded** again) by a double tap. When the coding is finished and all the sketches are folded, the final code looks almost identical to the traditional block-based language.

2.1.2 Rephrase blocks. Even though decomposition is mostly done by dividing a larger problem into smaller problems, programmers may sometimes want to express the problem in a different way without dividing it. To support these cases, we added **rephrase blocks**. Rephrase block is a block that only contains a sketch area with the same type of itself.

2.2 Multi-Layer Structure

In addition, the representation of block combinations also needed to be modified. In conventional block-based programming, a block is combined by being **inserted** into a hole or dent of another block, and therefore all blocks reside on a single plane. However, attachment slots cannot be represented to be visually blank in Sketchnote because they are now used as sketch areas. Blocks in Sketchnote are therefore **stacked** on other blocks like a pyramid as shown in Figure 1(c).

This multi-layer structure represents the hierarchical structure of the code. Since understanding problem decomposition is related to understanding various layers of abstraction [4], this structure might help novice programmers grasp the concept of problem decomposition. It also adds a new picture-drawing metaphor—drawing rough sketches at the bottom and then gradually overlaying detailed strokes and colors on top of them—to the coding process in block-based programming. Most importantly, this new metaphor does

not undermine the original Lego brick metaphor of block-based programming since Lego is supposed to be "stacked".

2.2.1 Level selection. To better support the multi-layer metaphor, we also added the layer level-selection feature. When a layer in the layer selection panel is pressed, only layers from level 1 (lowest layer) to the selected level are shown, and the ones above become transparent. This enables the decomposition process to be visually checked step by step, from the bottommost layer showing the complete flow of the program as an abstraction to the topmost layer showing the final code.

2.2.2 Viewpoint switching. The viewpoint-switching feature allows blocks to be displayed in two different modes: the default top view and an exploded isometric view. The exploded mode enables programmers to get an overview of the overall structure at a glance. This mode focuses on code viewing rather than editing, and we believe that it provides an intuitive and interesting way to represent the principle of Sketchnote.

3 IMPLEMENTATION AND TESTING

Sketchnote was implemented with the Unity framework [6], version 2021.2.8f1. The system was mostly tested on 11-inch iPad Pros and Apple Pencils, while compatibility with Android tablets, Mac computers, and Windows computers was also checked.

We also conducted a small between-subject user study with 10 participants, aged 11 to 13. The study consisted of four robot programming tasks, each with a 15-minute time limit, along with survey questionnaires and interviews. The study showed that using Sketchnote statistically reduced block attachment tries and tab conversions, which provides potential evidence that it can help programmers plan code in advance and thus reduce trial-and-errors. The interview also showed that the system was helpful for planning the code, remembering which block to put next, understanding the movement of the robot, and many others.

4 FUTURE WORKS

Based on the interim user study and in-lab heuristic evaluations, we are planning to improve Sketchnote further. We are specifically focusing on the position and visibility of sketches after blocks are combined on them. We are also checking usability issues of the level-selection and viewpoint-switching features, and planning improvements on them.

We are also planning a more in-depth user study to verify the effect of Sketchnote more rigorously. We could also deploy the system on app stores so that students over the world can use it. Other applications of sketching capability in block-based programming can also be explored: for example communication between teacher and student, compensating language differences, or documenting the code.

REFERENCES

- [1] Piraye Bayman and Richard E Mayer. 1988. Using conceptual models to teach BASIC computer programming. *Journal of Educational Psychology* 80, 3 (1988), 291.
- [2] John Maloney, Mitchel Resnick, Natalie Rusk, Brian Silverman, and Evelyn Eastmond. 2010. The scratch programming language and environment. *ACM Transactions on Computing Education (TOCE)* 10, 4 (2010), 1–15.

- [3] Tanya J McGill and Simone E Volet. 1997. A conceptual framework for analyzing students' knowledge of programming. *Journal of research on Computing in Education* 29, 3 (1997), 276–297.
- [4] Orna Muller, David Ginat, and Bruria Haberman. 2007. Pattern-oriented instruction and its influence on problem decomposition and solution construction. In *Proceedings of the 12th annual SIGCSE conference on Innovation and technology in computer science education*. 151–155.
- [5] Yizhou Qian and James Lehman. 2017. Students' misconceptions and other difficulties in introductory programming: A literature review. *ACM Transactions on Computing Education (TOCE)* 18, 1 (2017), 1–24.
- [6] Unity Technologies. 2005. *Unity*. <https://unity.com/>
- [7] David Weintrop. 2019. Block-based programming in computer science education. *Commun. ACM* 62, 8 (2019), 22–25.
- [8] Zhen Xu, Albert D Ritzhaupt, Fengchun Tian, and Karthikeyan Umapathy. 2019. Block-based versus text-based programming environments on novice student learning outcomes: A meta-analysis study. *Computer Science Education* 29, 2-3 (2019), 177–204.